udp UNIVERSIDAD DIEGO PORTALES

Torneo concurrente de Komikan

Sistemas Operativos – Tarea 2

Universidad Diego Portales

Lukas Diaz y Luciano Olivieri

1. Diseño del esquema de sincronización

El programa utiliza un proceso con hilos POSIX para clientes, emparejador, jueces y fichas. Los hilos comparten la memoria del proceso. Cada mesa activa tiene un juez y hasta trece hilos de fichas: uno para el puma y doce para los perros. El tablero contiene 28 casillas y un contador de capturas. Las conexiones y los saltos en línea recta se calculan al iniciar y después solo se consultan.

El mutex del servidor protege la fila de jugadores, la asignación de mesas y los contadores generales. El emparejador asigna los dos primeros clientes cuando hay una mesa libre. Si falta pareja o capacidad, espera en una variable de condición, sin busy waiting. Se reservan como máximo $\min(M, \lfloor N/2 \rfloor)$ mesas. Al terminar una partida, su mesa puede reutilizarse; si sobra un jugador, se informa y finaliza su espera.

Cada mesa tiene un mutex que protege el tablero, el resultado y la cola de solicitudes. La comunicación sigue el esquema productor–consumidor: las fichas producen pedidos y el juez los retira de una cola circular FIFO. Cada ficha mantiene como máximo un pedido pendiente; por eso, una cola con trece posiciones alcanza para todas. El juez espera si no hay pedidos y cada ficha espera en su propia condición hasta recibir una respuesta.

Solo el juez modifica el tablero durante la partida. La validación, el movimiento, la captura y la revisión del ganador se realizan en la misma sección crítica. Si dos fichas proponen el mismo destino, el segundo pedido se rechaza cuando la casilla ya está ocupada. También se rechazan pedidos de fichas capturadas. El puma gana con seis capturas; los perros, cuando el puma no tiene pasos ni saltos legales.

Las esperas usan `while` para volver a comprobar el estado después de despertar. `pthread_cond_wait` libera el mutex al bloquear el hilo y lo recupera antes de continuar. Para los retardos se utiliza `pthread_cond_timedwait`, una variante que permite esperar hasta un plazo y despertar antes si termina la partida. Los plazos y las mediciones usan `CLOCK_MONOTONIC`, mediante `clock_gettime` y `pthread_condattr_setclock`, para que un cambio de la hora del sistema no altere las esperas. El retardo configurado está entre 50 y 300 ms; la ejecución efectiva también depende del scheduler.

2. Riesgos de concurrencia

Deadlock. Si un hilo toma el mutex del servidor y espera el de una mesa, mientras otro los toma al revés, puede aparecer espera circular. Cuando se necesitan ambos, el orden es servidor y luego mesa. El juez libera el mutex de la mesa antes de esperar a las fichas con `pthread_join`, porque ellas lo necesitan para salir. Al reutilizar una mesa, se espera al juez anterior después de que este la haya marcado libre y liberado el mutex del servidor.

Starvation. La fila de clientes respeta el orden de llegada y la cola del juez atiende primero los pedidos más antiguos. Una ficha no puede llenar la cola porque espera su respuesta antes de enviar otra solicitud. Esto evita adelantar pedidos nuevos sobre los ya ingresados, pero no garantiza una espera acotada para obtener CPU o adquirir un mutex. Esos tiempos dependen también del scheduler y de la implementación de los bloqueos.

Livelock. Los movimientos rechazados no generan reintentos inmediatos: la ficha vuelve a realizar su espera temporizada antes de proponer otra jugada. El juez retira y resuelve cada pedido, sin reenviarlo continuamente entre hilos. Esto evita ese patrón de reintentos sin progreso. No garantiza que todas las partidas duren un tiempo limitado, pues las reglas no establecen un máximo de movimientos.

3. Evaluación y justificación del diseño

Los hilos permiten compartir directamente el estado del torneo. El mutex por mesa permite concurrencia entre partidas y paralelismo si hay núcleos disponibles. Dentro de una mesa, las modificaciones se realizan una por una. Esto simplifica las capturas, que cambian varias casillas y un contador, aunque concentra la atención de pedidos en un solo juez. Crear un hilo por cliente facilita representar su espera y abandono, pero aumenta el consumo de memoria y recursos cuando crece N.

Funciones de apoyo. Cada ficha utiliza un estado propio de `xorshift32` para elegir retardos y jugadas. Es un generador pseudoaleatorio pequeño basado en desplazamientos y XOR; evita compartir un estado aleatorio modificable entre fichas. La semilla se combina con los identificadores de partida y ficha, evitando el estado cero. No tiene un objetivo criptográfico ni garantiza partidas idénticas, porque el orden de ejecución puede cambiar.

El juez copia el tablero bajo su mutex y lo libera antes de imprimir. `flockfile` y `funlockfile` protegen el bloque de salida para que las impresiones de varias mesas no mezclen sus líneas. No reemplazan al mutex del tablero. La impresión permite que otras fichas encolen pedidos, pero mantiene ocupado al juez hasta que termine de escribir.

Para leer la configuración, `getline` obtiene líneas completas sin fijar un tamaño de buffer. `strtol` y `strtoul` permiten comprobar conversiones, rangos y caracteres sobrantes. `calloc` reserva estructuras inicialmente en cero y `free` libera su memoria. Se registran los hilos creados y los bloqueos inicializados para limpiar solo los recursos válidos si la preparación falla.

Abandono y señales. El cliente configurado para abandonar usa `pthread_exit` al vencer su plazo. `pthread_cleanup_push/pop` establece una rutina que registra el abandono cuando corresponde y libera el mutex del servidor. `pthread_setcancelstate` deshabilita temporalmente la cancelación mientras se prepara esa limpieza y se registra al cliente. El juez concede la victoria al rival y espera a las fichas antes de liberar la mesa. Este mecanismo maneja la salida del cliente; no aísla un segmentation fault, que afecta al proceso completo.

`pthread_sigmask` bloquea las señales antes de crear los hilos, que heredan esa máscara. Un hilo dedicado las recibe con `sigwait`. Así, el cierre usa mutexes desde un hilo normal, sin hacerlo dentro de un manejador asíncrono. Ante `SIGINT` o `SIGTERM`, se detienen las nuevas asignaciones y se despierta a los participantes con `pthread_cond_broadcast`. Luego se esperan los hilos, se destruyen los bloqueos y se libera la memoria. En el término normal, `pthread_kill` envía `SIGUSR1` al hilo de señales para que salga de su espera; solo se acepta como aviso de salida cuando el servidor ya marcó el fin. No se crean procesos hijos ni objetos IPC persistentes.

Pruebas y rendimiento. El commit `559b824` compiló en Linux con `-Wall -Wextra -Werror -std=c17 -pthread`, sin advertencias. Pasaron las pruebas de reglas, configuración inválida, jugador sin pareja, límite de mesas, abandono y cierre con `SIGINT`. Una ejecución de `./torneo 8 2 config.txt` produjo:

Medida	Resultado
Partidas / jugadores finalizados / máximo de mesas	4 / 8 / 2
Movimientos aceptados / rechazados	714 / 4
Tiempo total	6,73 s
Throughput: partidas/s / movimientos aceptados/s	0,59 / 106
Latencia media / máxima	2,787 / 184,737 ms

El throughput mide el trabajo procesado por segundo. La latencia va desde el ingreso del pedido a la cola hasta su resolución; excluye el retardo previo y la espera por el mutex antes de encolar. Los rechazos son propuestas que dejaron de ser válidas antes de atenderse. Son resultados de una corrida: el scheduler, la carga y la impresión afectan los tiempos. Las pruebas comprueban estos casos, no todas las ejecuciones posibles.